Мультиплексирование ввода-вывода

Илья Кокорин

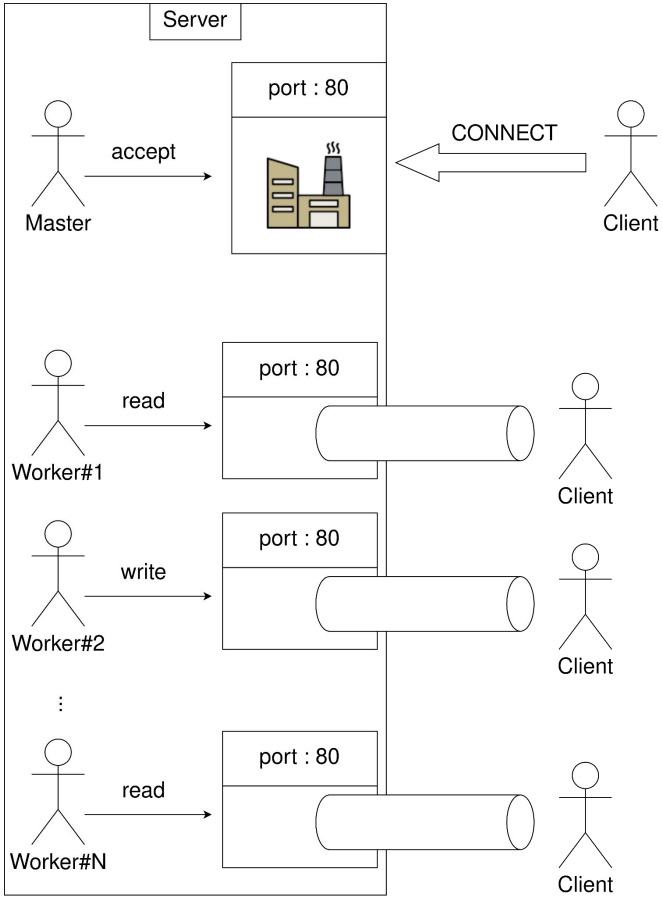
kokorin.ilya.1998@gmail.com

TCP: многопоточный сервер

- Обрабатываем каждого клиента в отдельном потоке

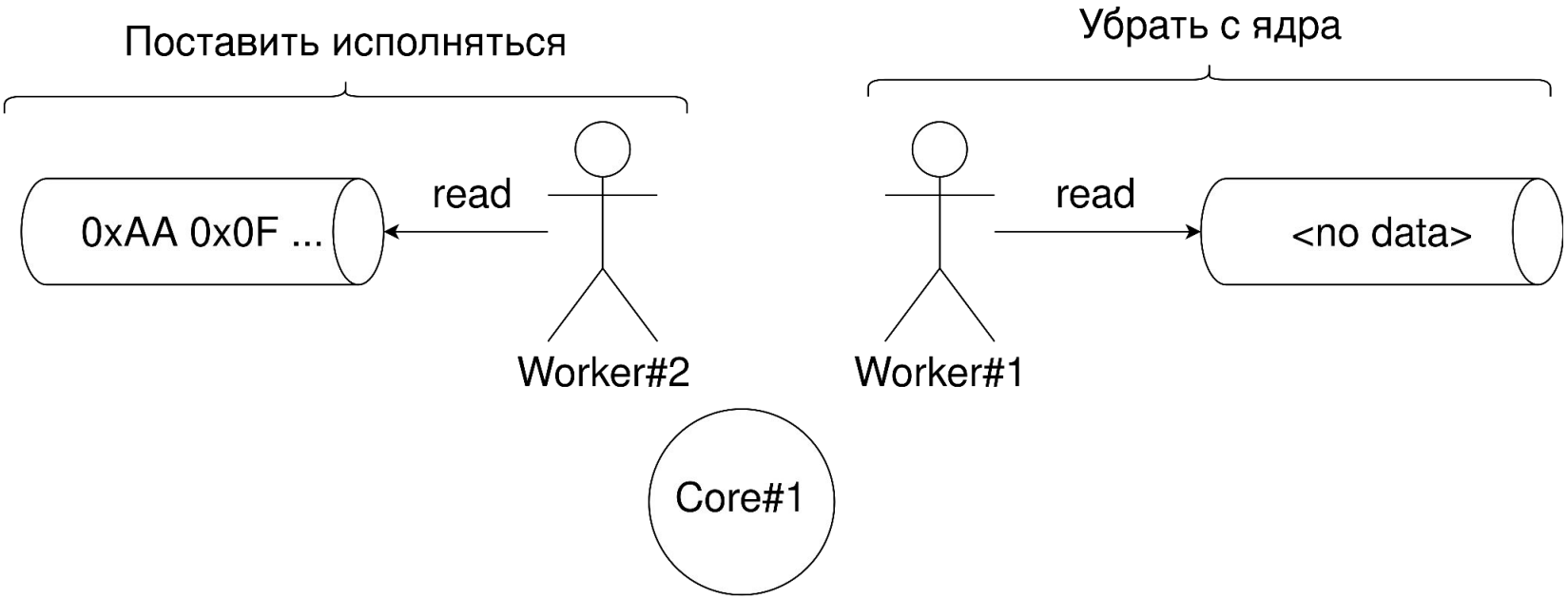
```
fd = socket(IPPROTO_TCP)
bind(fd, host, port)
listen(fd, bqs)
while true:
    conn := accept(fd)
    thread {
        process_client(conn)
        close(conn)
    }
close(fd)
```

TCP: многопоточный сервер



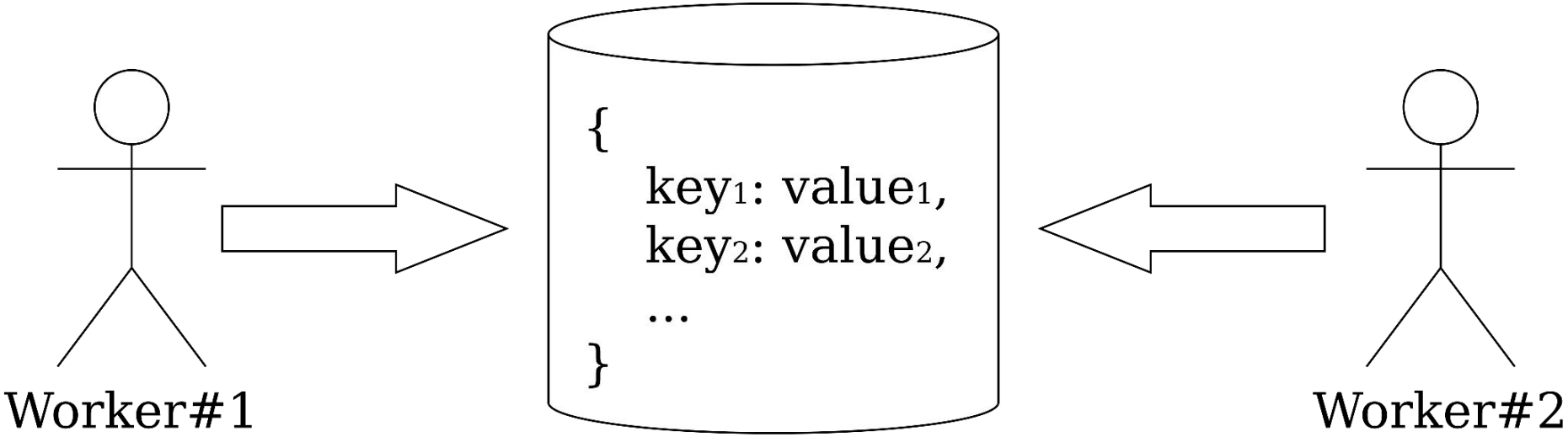
Цена потока

- Поток занимает оперативную память
- context switch не бесплатные



Цена потока

- Сложность обновления общего состояния
- Вы знаете это с параллельного программирования

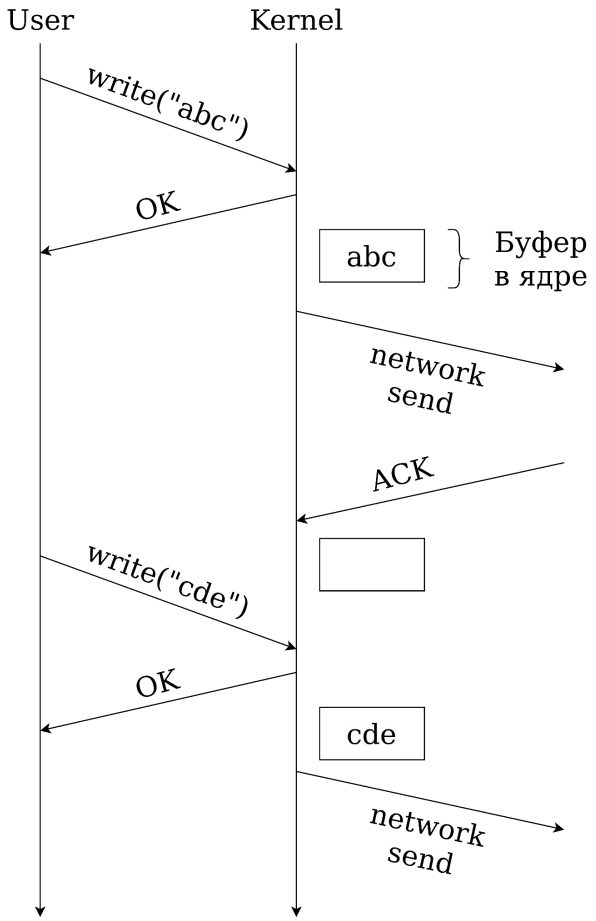


Неблокирубщие сокеты

- `fcntl(fd, O_NONBLOCK)`
- `data, ok := try_read(fd, size)`
  - `ok = false` если данных пока нет
- `ok := try_write(fd, buf, offset, size)`
  - `ok = false` если записать не получается
  - Если сетевой стек ОС занят отправкой других данных
- `conn, ok := try_accept(fd)`
  - `ok = false` если новый клиент пока не пришёл
- И т.д.

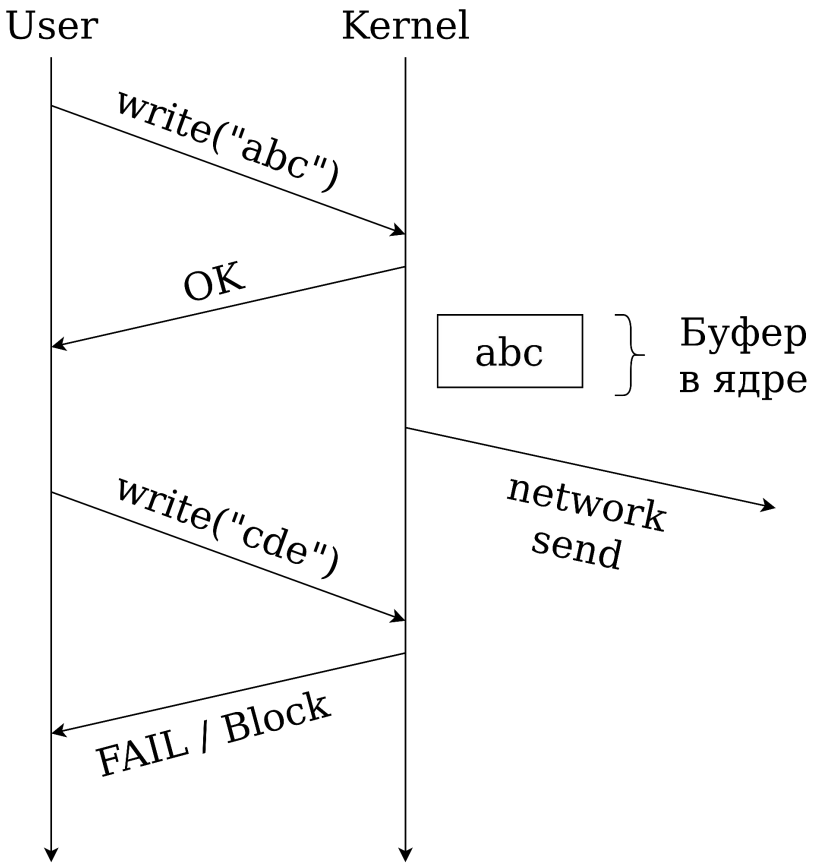
Запись в TCP сокет

- У TCP сокета под капотом есть буфер
- В него записывается очередной кусок данных
- ОС отправляет его
- И ждёт подтверждения получения
- Перепосылает, если нужно
- Освобождает буфер при получении подтверждения
- В него можно записывать новые данные



Запись в TCP сокет

- В буфере может не быть места
- Потому что не пришло подтверждение приёма предыдущей порции данных
- Новую порцию данных записать в буфер нельзя
- Синхронный сокет заблокируется
- Асинхронный — вернёт ошибку



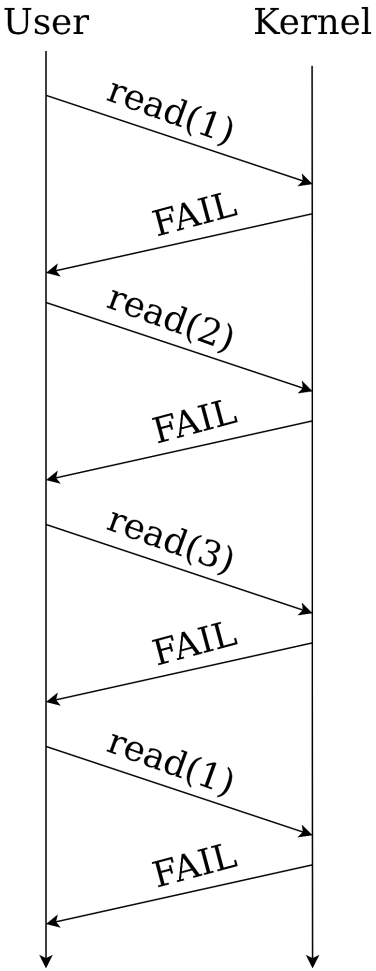
Неблокирубщие сокеты

- Пробуем работать с каждым сокетом по кругу
- Переходим к следующему, если не получилось

```
fd := socket(IPPROTO_TCP | O_NONBLOCK)
bind(fd, host, port)
listen(fd, bqs)
state := initial_state()
conns := []
while true:
    conn, ok := try_accept(fd, O_NONBLOCK)
    if ok:
        conns.append(conn)
    for conn in conns:
        data, ok := try_read(conn)
        if ok:
            state ← change_state(state, data)
    ...
```

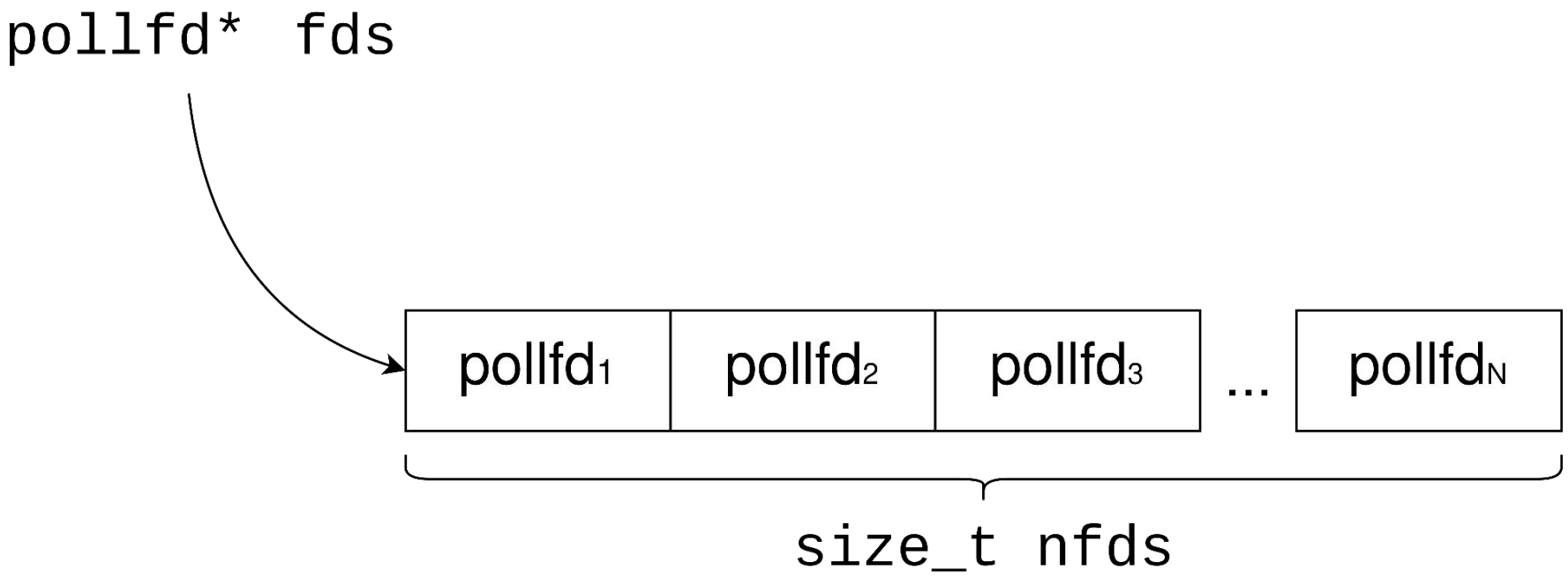
Неблокирубщие сокеты

- Активное ожидание
- Поллинг ОС
- Впустую тратим CPU
- Хочется чтобы ОС сама оповещала нас, когда в интересующем нас сокете появятся данные
  - Или возможность их записать
  - Или новый клиент
  - И т.д.



poll

- Системный вызов
- `int poll(pollfd* fds, size_t nfds, int time);`



poll

- Структура `pollfd` состоит из трёх полей

```
/* Data structure describing a polling request.  */
struct pollfd
  {
    int fd;                     /* File descriptor to poll.  */
    short int events;           /* Types of events poller cares about.  */
    short int revents;          /* Types of events that actually occurred.  */
  };
```

- `fd` это номер файлового дескриптора, на котором мы хотим ждать возможности выполнения операции
  - Чтения/записи/нового клиента
- Два оставшихся поля это битовые маски

poll: битовые маски

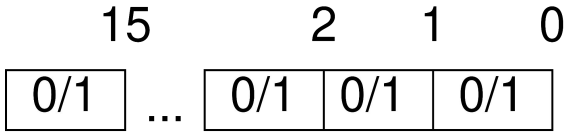
- `short` состоит из 16 бит
- Каждый бит отвечает за какую-то операцию
  - 0 — за чтение и приём новых клиентов
  - 2 — за запись
  - 3 — за ошибки
  - 4 — за закрытие соединения
  - И т.д.

15		2	1	0
0/1	...	0/1	0/1	0/1

- Мы можем хотеть получить какое-то множество событий
  - Говорим об этом, ставя биты в `events`
  - `pollfd.events = (1 << 0) | (1 << 2)`
  - `pollfd.events = POLLIN | POLLOUT`

poll: битовые маски

- `short` состоит из 16 бит
- Каждый бит отвечает за какую-то операцию
  - 0 — за чтение и приём новых клиентов
  - 2 — за запись
  - 3 — за ошибки
  - 4 — за закрытие соединения
  - И т.д.



- Аналогичным образом в `revents` ядро возвращает, какие события произошли с дескриптором
  - `(pollfd.revents & (1 << 2)) > 0`
  - `(pollfd.revents & POLLOUT) > 0`

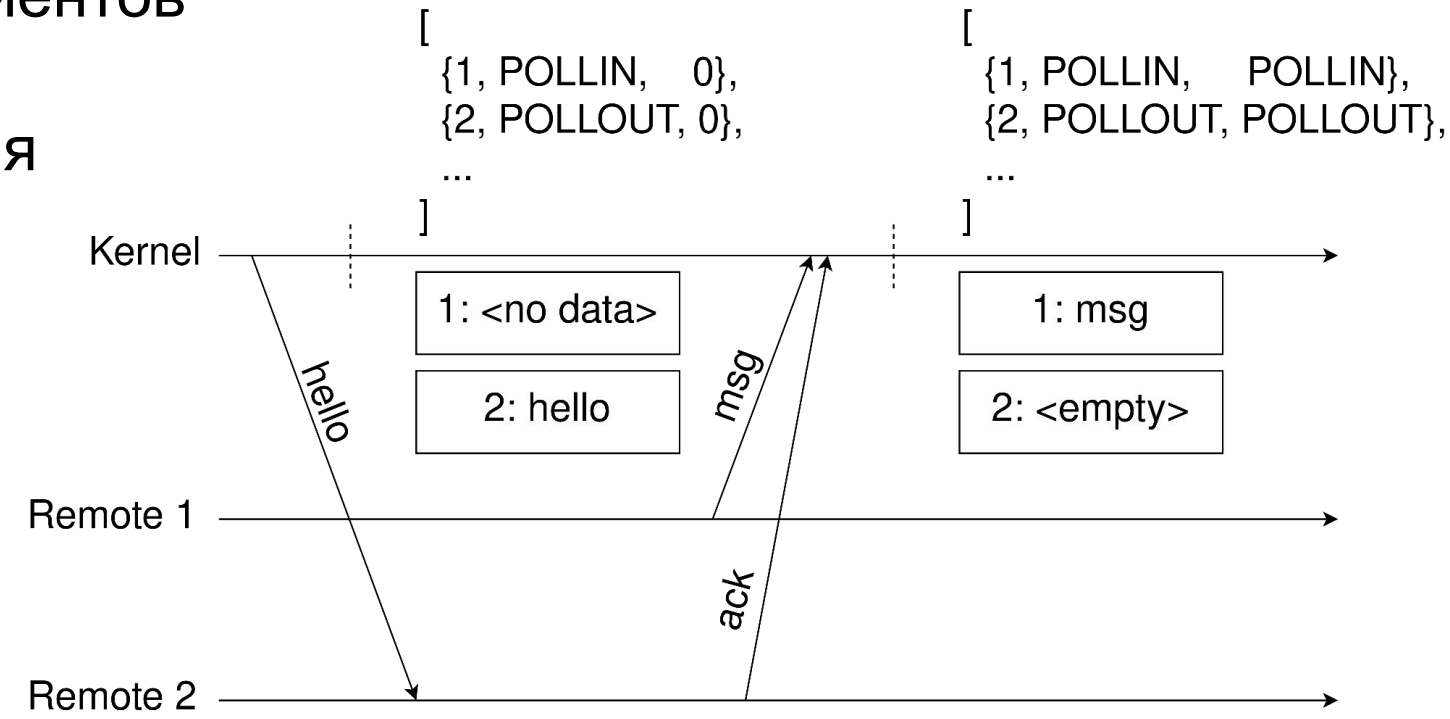
poll

```
fds = []
for conn in conns:
    cur_flag = 0
    if should_read:
        cur_flag |= POLLIN
    if should_write:
        cur_flag |= POLLOUT
    if cur_flag == 0:
        continue
    fds.append({ fd = cur_fd, events = cur_flag, revents = 0 })

poll(fds)
for cur_fd in fds:
    if cur_fd.revents & POLLIN != 0:
        process_read(cur_fd.fd)
    if cur_fd.revents & POLLOUT != 0:
        process_write(cur_fd.fd)
```

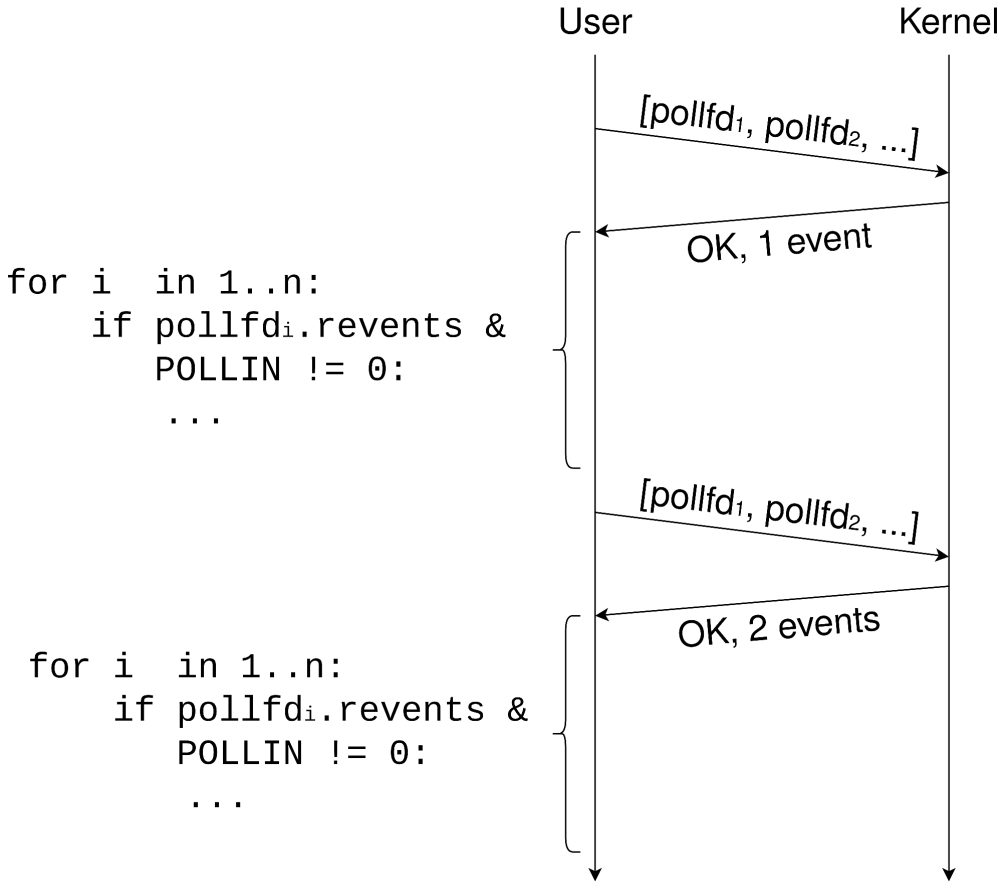
poll: внутреннее устройство

- Реагирует на освобождение буферов для записи
- Получение данных
- Новых клиентов
- И прочие сообщения из сети

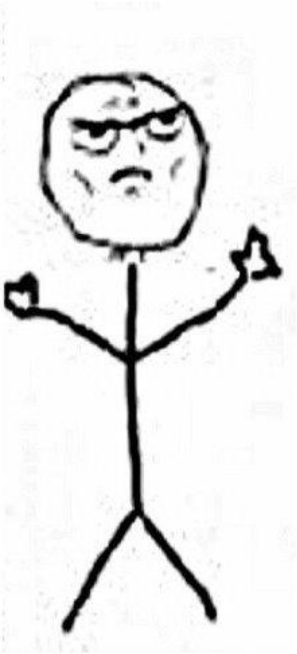


poll: проблемы с масштабируемостью

- Очень много времени тратится на обход элементов массива с `revents = 0`



Что почитать

- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]
- [ДАННЫЕ УДАЛЕНЫ]

спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание
спасибо за внимание